LifeTech
ITS Academy

CORSO DI TECNICO SUPERIORE PER LO SVILUPPO E LA GESTIONE DI SOLUZIONI DI INFORMATICA BIOMEDICA (TIB) 2024-2026

Codice progetto: 2024/5173/0

UF 9.1 PROGRAMMAZIONE AD OGGETTI IN JAVA

DOCENTE: ALESSANDRO BISONTIN, SIMONE KRESEVIC

DATA: 02/03/2026

FUTURA
Finanziato dall'Unione europea NextGenerationEU
Ministero dell'Istruzione e del Merito
LA SCUOLA PER L'ITALIA DI DOMANI
PNRR ISTRUZIONE

Esercitazione: Sistema Triage Pronto Soccorso

Durata: 3 ore — OOP + Stringhe + Scanner

UF9.1 – Programmazione ad Oggetti in Java

Consegna

Contesto

Realizzare un sistema di gestione triage per il pronto soccorso di un ospedale.
Il sistema deve gestire l'accettazione di pazienti e la registrazione delle visite mediche.

Il sistema funziona tramite un menu da console (Scanner) e gestisce un paziente alla volta.

Argomenti richiesti

Classi e oggetti, costruttori, incapsulamento (private + getter/setter), ereditarietà,
classi astratte, polimorfismo, override, super, this, casting, metodi String, Scanner, printf.

Tempo a disposizione: 3 ore

Contesto

Il **triage** è una procedura infermieristica rapida e sistematica utilizzata nel Pronto Soccorso per valutare i pazienti all'arrivo, assegnando una priorità di trattamento (codice) basata sulla gravità clinica e sul rischio evolutivo, piuttosto che sull'ordine di arrivo.

L'obiettivo è identificare i casi critici e ottimizzare le cure dove un infermiere esperto valuta i parametri vitali e i sintomi per classificare il paziente assegnando dei codici di priorità (5 livelli).

- Rosso (Emergenza): Pericolo di vita immediato.
- Arancione (Urgenza): Paziente con patologie gravi a rischio evolutivo.
- Azzurro (Urgenza differibile): Condizioni stabili con rischio evolutivo.
- Verde (Urgenza minore): Patologie di modesta entità.
- Bianco (Non urgenza): Problemi non urgenti.

Requisiti funzionali

1. Registrare un paziente con: codice fiscale (16 caratteri), nome, cognome, codice colore triage (Bianco, Verde, Giallo, Rosso)
2. Validare il codice fiscale: deve avere esattamente 16 caratteri e non essere vuoto. Se non valido, stampare un messaggio di errore e chiedere di reinserirlo
3. Creare una visita medica per il paziente. Esistono 3 tipi di visita: VisitaGenerica, VisitaOrtopedica, VisitaCardiologica (ereditarietà)
4. Ogni tipo di visita ha un metodo descriviVisita() diverso (polimorfismo/override). La classe base Visita è astratta
5. Stampare la scheda completa del paziente con output formattato (printf) includendo tutti i dati e i dettagli della visita
6. Implementare un metodo calcolaPriorita() nella classe Paziente che, in base al colore triage e al tipo di visita, restituisce una stringa di priorità
7. Menu interattivo da console che permette di: registrare un paziente, assegnare una visita, stampare la scheda, calcolare priorità, uscire

Diagramma delle Classi UML

Paziente

```
- codiceFiscale: String
- nome: String
- cognome: String
- coloreTriage: String
- visita: Visita

+ Paziente(cf, nome, cognome, colore)
+ getCodiceFiscale(): String
+ getNomeCompleto(): String
+ getColoreTriage(): String
+ getVisita(): Visita
+ setVisita(v: Visita): void
+ calcolaPriorita(): String
+ stampaScheda(): void
```

Main

```
+ main(args: String[]): void
+ mostraMenu(): void
```

"se ha, ha una"
0..1

<<abstract>>

Visita

```
- data: String
- medico: String
- diagnosi: String

+ Visita(data, medico, diagnosi)
+ getData(): String
+ getMedico(): String
+ getDiagnosi(): String
+ descriviVisita(): String
{abstract}
```

extends (ereditarietà)

VisitaGenerica

```
- reparto: String

+ VisitaGenerica(...)
+ getReparto(): String
+ descriviVisita(): String
```

VisitaOrtopedica

```
- parteCorpo: String

+ VisitaOrtopedica(...)
+ getParteCorpo(): String
+ descriviVisita(): String
```

VisitaCardiologica

```
- frequenzaCardiaca: int
- pressioneSist: int
- pressioneDiast: int

+ VisitaCardiologica(...)
+ getFrequenza(): int
+ descriviVisita(): String
```

Dettagli implementativi

Classe Paziente

- Tutti gli attributi private con getter
- getNomeCompleto() restituisce cognome + " " + nome in maiuscolo (toUpperCase)
- L'attributo visita può essere null (non ancora assegnata)
- stampaScheda() usa printf per formattare l'output

Metodo calcolaPriorita()

- Se nessuna visita assegnata → "NON VISITATO"
- Rosso + Cardiologica → "CODICE 1 - EMERGENZA"
- Rosso + altro tipo → "CODICE 2 - URGENTE"
- Giallo → "CODICE 3 - PRIORITARIO"
- Verde/Bianco → "CODICE 4 - STANDARD"

Classe astratta Visita

- Ha un costruttore che riceve data, medico, diagnosi
- descriviVisita() è abstract → ogni sottoclasse deve implementarla con @Override
- Getter per tutti gli attributi (ereditati dai figli tramite super)
- Non può essere istanziata direttamente

Le 3 sottoclassi di Visita

- VisitaGenerica: ha un attributo reparto (String). descriviVisita() restituisce diagnosi + reparto
- VisitaOrtopedica: ha parteCorpo (String). descriviVisita() include la parte interessata
- VisitaCardiologica: ha frequenzaCardiaca (int), pressioneSist e pressioneDiast (int). descriviVisita() include i parametri vitali
- **Tutte chiamano super() nel costruttore!**

Validazione con i metodi String

Usate i metodi delle stringhe per validare gli input dell'utente. Ecco cosa dovete controllare:

Codice Fiscale	Deve avere esattamente 16 caratteri	`trim().length() == 16`
Codice Fiscale	Non deve essere vuoto	`!trim().isEmpty()`
Codice Fiscale	Convertire in maiuscolo	`toUpperCase()`
Colore Triage	Deve essere Rosso, Giallo, Verde o Bianco	`equalsIgnoreCase("rosso")`
Nome / Cognome	Non devono essere vuoti	`!trim().isEmpty()`
Data visita	Formato gg/mm/aaaa (10 caratteri)	`trim().length() == 10`
Nome medico	Prima lettera maiuscola nel display	`substring(0,1).toUpperCase() + substring(1)`

Esempio di output atteso

Questo è un esempio di come deve apparire la scheda paziente (stampaScheda con printf):

```
╔═══════════════════════════════════════════╗
║         SCHEDA PAZIENTE PRONTO SOCCORSO   ╒
╠═══════════════════════════════════════════╣
╒ Codice Fiscale: RSSMRA85M01H501Z          ╒
╒ Paziente:       ROSSI MARIO               ╒
╒ Triage:         ROSSO                     ╒
╠═══════════════════════════════════════════╣
╒ VISITA CARDIOLOGICA                       ╒
╒ Data:           15/01/2026                ╒
╒ Medico:         Dott. Bianchi             ╒
╒ Diagnosi:       Sospetta aritmia          ╒
╒ Freq. cardiaca: 120 bpm                   ╒
╒ Pressione:      150/95 mmHg               ╒
╠═══════════════════════════════════════════╣
╒ PRIORITÀ:       CODICE 1 - EMERGENZA      ╒
╚═══════════════════════════════════════════╝
```

Come organizzare le 3 ore

0:00 – 0:30	Leggere la consegna e l'UML. Creare i file .java per ogni classe.
0:30 – 1:00	Implementare Visita (astratta) e le 3 sottoclassi. Testare descriviVisita().
1:00 – 1:30	Implementare la classe Paziente con tutti i getter, stampaScheda() e calcolaPriorita().
1:30 – 2:15	Implementare il Main con il menu Scanner. Validare gli input con i metodi String.
2:15 – 2:45	Testare tutti i casi: diversi colori triage, diversi tipi di visita, input errati.
2:45 – 3:00	Rifinire l'output printf, aggiungere commenti al codice, verifica finale.

Suggerimenti:

Compilate spesso! Non scrivete tutto e poi compilate alla fine.
Testate ogni classe appena creata.